

杨大春

Go 后端开发 | 从 Outbox 事务消息到分布式缓存：把高并发 Feed 流的每一跳做成可验证的系统

随时到岗
电话 13425922492
邮箱 2033231795@qq.com

教育经历

西南科技大学 四川省人民政府与教育部共建高校 四川省人民政府与国家国防科技工业局共建高校 国家重点建设的西部 14 所高校之一 办学历史源起 1952 计算机科学与技术 / 本科	2022.09 - 2026.07
- 主修课程：数据结构与算法、计算机操作系统、数据库原理及应用、C++ 程序设计、微机原理及技术。 - 校园经历：加入本校移动互联网实验室，参与合作开发流程，积累多人协作与工程化开发实践。	

实习 / 工作经历

短视频 Feed 流系统 个人全栈项目 Go / Gin / MySQL / Redis / RabbitMQ / Vue 3	2026.04 - 2026.07
短视频 Feed 流系统 - 个人项目全链路 Owner 从 0→1 搭建 API+Worker 双进程异步架构，打通写链路异步化 - 最终一致性 - 多级缓存 - 冷热分离 - 分片上传	
背景：	- 基于 Go + Vue 3 全栈实现的短视频 Feed 流系统，覆盖账号鉴权、视频发布、点赞、评论、关注、多路 Feed 流（最新/点赞/关注/热度）与通知等核心模块。 - 架构以 Redis 多级缓存、RabbitMQ 事件驱动异步、分片上传与断点续传、SSE 实时推送为骨架，围绕高并发、数据一致性与容灾降级设计。
指标与效果：	- 压测闭环（JMeter 100 并发 × 60s，混合 latest/detail/popularity/likes_count 4 接口）：定位 GORM 默认 MaxIdleConns=2 引发的短连接风暴（本机 1.2 万 + TIME_WAIT），调优连接池（MaxOpen=100/MaxIdle=50/Lifetime=1h）后，100 并发吞吐从 612 QPS、错误率 6.66% 提升至 4162 QPS、0 错误，feed/latest P99 53ms、video/detail P99 35ms；50 并发达 4662 QPS。 - 写链路异步化：点赞、评论、关注、热度更新写操作经 RabbitMQ Topic Exchange 异步分发，接口同步压力大幅降低；MQ 通道故障时自动直写 MySQL/Redis 兜底，消息中间件不可用场景下核心业务数据不丢失。 - 读链路三级缓存：Feed 视频实体采用 L1 本地缓存（5s TTL）→ L2 Redis（1h TTL）→ L3 MySQL；缓存未命中时 singleflight 合并并发回源，配合 SETNX 分布式锁只放行 1 个回源请求、其余等待--同一失效 key 的并发请求只打 1 次数据库，防缓存击穿。 - 时间线热窗口：全局时间线以 Redis ZSET（时间戳为 Score）构建，仅保留最新 1000 条热数据，热数据读取直接命中 ZSET；自动裁剪避免热窗口无界膨胀。 - 热度榜快照：按分钟分桶写入 ZSET，查询时 ZUNIONSTORE 聚合最近 60 个分钟窗生成只读快照（2 分钟有效），翻页复用同一 as_of 榜单版本，消除实时排名变化导致的翻页重复与跳页；点赞榜以（点赞数, 视频 ID）复合游标唯一化排序键，并列点赞数下分页不重不漏。 - 通知实时触达：点赞/评论/关注事件经 MQ 分发至 SSE Hub（internal/ssehub）实时推送给目标用户，并过滤用户自身触发的事件，前端无需轮询。
我的职责：	- 大文件上传：5MB 分片 + 逐分片 SHA-256 校验防篡改，Redis 维护 24 小时有效期上传会话与已传分片索引，支持同一文件断点续传；合并完成后自动清理临时分片与缓存会话。 - 独立完成 API + Worker 双进程架构搭建：写操作经 RabbitMQ Topic Exchange 异步解耦削峰；设计 MQ 故障降级策略，通道发送失败自动直写 MySQL/Redis 兜底。 - 实现 Outbox 事务消息保证分布式最终一致性：视频发布在同一本地事务中写入业务数据 + Outbox 消息记录，定时任务轮询未投递记录发送 MQ、投递成功删除记录，规避“业务入库成功但事件丢失”的不一致问题。 - 实现独立 Timeline 消费者：消费视频发布事件写入 Redis ZSET 构建全局时间线，以时间戳为 Score 排序并自动裁剪至近 1000 条热数据。 - 实现冷热分离与无抖动分页：ZSET 热窗口承接热数据读取，游标越过水位线自动回源 MySQL 拼接冷数据，冷区结果不回写热窗口；热度榜分钟分桶 + ZUNIONSTORE 快照 + as_of 版本化翻页。 - 实现三级缓存与降级链路：singleflight 合并回源 + SETNX 分布式锁防击穿；Redis 故障时核心查询自动回退 MySQL，保障缓存层故障下服务可用。 - 搭建 JMeter 压测闭环并修复连接池短板：定位 GORM 默认连接池参数导致的高并发短连接风暴，完成连接池调优与修复前后吞吐/错误率/P99 对比验证。
技术关键词：	Go / Gin / MySQL / Redis / RabbitMQ / Outbox 事务消息 / Topic Exchange / SingleFlight / SETNX 分布式锁 / ZSET 时间线 / ZUNIONSTORE 快照 / SSE 实时推送 / 分片上传 / 断点续传 / SHA-256 校验 / JMeter 压测 / GORM 连接池调优

分布式缓存系统 Lcache - 全链路 Owner | 从 0→1 实现一致性哈希路由 - 桶级分段锁 - etcd 服务发现 - gRPC 节点同步

- 背景：
- 基于 Go 实现的高性能分布式内存缓存系统，支持 LRU / LRU-2 多种缓存淘汰策略、TTL 过期与基于 etcd 的分布式协调，面向分布式环境下的高效数据共享与访问。
- 指标与效果：
- 负载均衡：实现自适应一致性哈希，支持虚拟节点与动态负载均衡，节点负载偏差超过 25% 时自动调整副本数，保证数据均匀分布并减少节点增减时的数据迁移。
 - 并发吞吐：BKDR 哈希分段锁 + 两级缓存结构将锁粒度降至桶级，减少锁争用、提升高并发吞吐；SingleFlight 请求合并使同一 key 的并发请求仅回源 1 次，防缓存击穿、降低后端压力。
- 我的职责：
- 可用性：基于 etcd 租约续约 + watch 增量监听实现节点自动注册/摘除，集成 gRPC 健康检查，节点上下线对客户端透明。
 - 实现 LRU、LRU-2 两种缓存淘汰算法与 Key 自定义 TTL 定时过期清理，并针对热点重复访问场景优化 LRU-2 淘汰逻辑。
 - 实现 BKDR 哈希分段锁与两级缓存结构，将锁粒度从全局降低到桶级，减少锁争用。
 - 基于 etcd 实现服务注册与发现：租约续约维持节点在线状态，watch 增量监听节点变化实现自动注册/摘除，并集成 gRPC 健康检查。
 - 基于 gRPC + Protobuf 实现节点间通信：写入/删除操作异步同步到对端，以标记防止同步循环，保证分布式环境下的数据一致性。
- 技术关键词： Go / etcd / gRPC / Protobuf / 一致性哈希 / 虚拟节点 / LRU-2 / TTL / BKDR 分段锁 / SingleFlight / 租约续约 / watch 增量监听 / 健康检查

技术项目与沉淀

工程技能图谱 | Go 后端技术栈

- 语言：熟悉 Go 语言开发，了解 GPM 调度模型与 GC 垃圾回收机制。
- MySQL：熟悉索引、锁、事务、存储引擎、MVCC 与日志体系，掌握 MySQL 查询优化方法。
- Redis：熟悉数据结构、持久化机制与缓存淘汰机制，理解缓存雪崩、穿透、击穿场景及解决方案。
- 框架：熟练使用 Gin、go-zero 等开发框架，能针对业务进行二次开发。
- 中间件：熟练使用 RabbitMQ，理解消息队列可靠投递原理与死信队列，能解决消息丢失与重复消费问题；熟悉 etcd 服务发现与配置管理。
- 工具：熟练使用 Git 进行团队协作开发，掌握 Linux 常用命令与脚本操作。